BA Besprechungsnotizen

Bjarne Jehn

April 2026

Besprechung am 03.09.

Nächste Schritte

- *no_minimizers* ist falsch, da es sich hierbei nur um die anzahl an unfrequenten minimizern handelt

Fragen

- im rshash repository wird häufig 'minimiser' statt 'minimizer' genutzt
- falscher Datentyp bei berechnung von $P_{\mathcal{M}}$? Verteilung "springt"
- wofür sind *cum* und *cum_skmers* bei der Berechnung der Minimizer stats nötig?
- lässt sich *get_uncovered_kmers* ohne die loops über 1-t umsetzten? So, dass dabei nicht direkt die kummulative Summer gezählt wird sondern die exakte Anzahl an k-meren mit minimizern dieser Frequenzen?
- nur eine idee: könnte es nicht interessant sein, den zusammenhang zwischen der frequenz eines minimizers und der länge der zugehörigen super kmere zu betrachten?
  Vermutung dazu: wenn gerade "kleine" minimizer besonders frequent sind, dann müssten diese fast alle anliegenden k-mere abdecken und somit zu langen super k-meren gehören.
  Ist das genau der zusammenhang zwischen p_K und p_M?

Besprechung am 26.08.

Nächste Schritte

- -l durch –level ersetzten
- gewichtetes minimierungs Ziel in Form von $\alpha \cdot runtime + \beta \cdot bpk_{total}$ oder ähnlichem definieren

- **Roadmap**:
  1. Verteilung auf mehreren Leveln berechnen und untersuchen
  2. Experimente durchführen
     (a) Server Zugang
     (b) $T_{access/select/rank/...R/S/...}$ messen
     (c) MINLP für verschiedene Genome laufen lassen und mit tatsächlichem Speicherbedarf & Laufzeit von RSHash vergleichen
     (d) (dabei Daten für ML Ansatz sammeln)

Fragen

- f_{avg}: müsste $p_K(m, f)$ normiert werden, da es sich um eine bedingte wahrscheinlichkeit handelt? - ja, ist sinnvoll
- $(f_{avg} + 1)/2)$ sinnvoll für lookup? - scheint so
- O Notation für $t_{...}$ sinnvoll? - wohl eher nicht -lieber messen
- rshash/source/build.cpp und ministats.cpp durchsprechen - besonders:
  - get uncovered skmers und freq minimizers

Besprechung am 12.08.

Nächste Schritte

- p_K muss gewichtet betrachtet werden - wenn viele kmere den gleichen minimizer teilen, dann gibt es viele vergleiche mit dem Text, welche teuer sind - deswegen $p_K * durschnittlicheFrequenzvonunfrequentminimizern * T_{SL}$, sonst unverändert durchschnittliche frequenz berechnen - durchschnittliche frequenz: $p_K(m, 1) * 1 + (p_K(m, 2) - p_K(m, 1)) * 2 + ...$ - done
- für mehrere Level wird der lookup in R relevant, da dieser für höhere Level mehrfach stattfindet - ca. 30ns für check in R, 35ns für check in S+ 10ns für jeden check in O, 20ns für H - die Zeiten genau zu modellieren, wäre für die Praxis sinnvoll
- pdf mit pushed, raus aus gitignore - done
- zu subsampling: entweder aus dem ganzen Text zufällig wählen, oder anteilig an der länge der einzelnen sequenz, da sonst ein bias zu den kürzeren teilsequenzen besteht

Fragen

- sinnvoll bpk als constraint nutzen: abhängigkeit von R und S berücksichtigen? (bei korrekter modellierung könnten dann tatsächlich exakte maximale bpk werte festgelegt werden) - einfach mal mitmodellieren und dann argumentieren, warum es nicht relevant ist - wäre aber cool für den vergleich
- ist es trivial, dass die parameter für steigende level non decreasing sind? - schon, dadurch könnte sich die Laufzeit verbessern lassen
- warum hat die repititivität des genoms einen einfluss auf die kompression, wenn doch eh alle duplikate entfernt werden?
- modellierte bpk Werte sind deutlich höher als jene im paper, woran könnte das liegen?
  - vllt daran, dass wiederholte kmere bereits entfernt wurden?
  - stammen die messwerte aus dem paper von vorbearbeiteten genomen ohne repititivität oder von unbearbeiteten genomen?
  - wie ist die länge de menschlichen genoms ohne repetitive sequenzen?
- chapter subsampling: wie strukturieren? Laufzeit und Fehler hängen von n, l ab aber auch die Wahl von n, l wird erst durch die Analyse von Laufzeit und Fehler begründet. Was soll ich zuerst einführen?
- Meinung/Tips zu include oder subfile
- gitignore....

Besprechung am 22.07.

Nächste Schritte

- Anmerkungen übernehmen - done
- Bennenung von minimizer länge m vs. minimizer min/M/..? - done
- frequency immer als funktion f(..) nutzen -done
- space consumption der Hashtabelle anpassen -done

Fragen

- Hashtable/Lookup time?
- warum werden binary constraints misachtet, wenn empirische Werte /100 geteilt werden? - numerische Fehler?

Besprechung am 08.07.

Nächste Schritte

- gurobi solver probieren - bisher nicht nötig

Fragen

- rshash/test/hashtable.cpp korrigieren: Zeile 72 und 91 jeweils "value" statt "kmer_value" und "value_rev" statt "kmer_value_rev"

Besprechung am 17.06.

Nächste Schritte

- ~~im ilp unterscheiden zwischen variablen und konstanten~~
- ~~erstmal nur ein level modellieren~~
- ~~ilp begründen schriftlich~~

Fragen

-

Besprechung am 10.06.

Nächste Schritte

- linearisiern
- oder einfach mal probieren
- für fixe werte mal durch einen solver laufen lassen und ergebnisse bewerten
- schonmal mit dem schreiben beginnen
- erst danach über code ausdenken
- git!

Fragen

- Sollte man die Space Consumption nicht optimieren sondern eher möglichst genau modellieren, um dann eine obere Grenze wie z.B. 8Gb setzten zu können?

Besprechung am 13.05.

Nächste Schritte

- git zugang

Fragen

- Ist die Annahme, dass die Minimizer-Verteilung in den höheren Leveln ähnlich wie in Level 1 ist allgemein korrekt? - höchst wahrscheinlich schon.
- Wie kann ich festellen, wie viele von 4^m möglichen Minimizern vorkommen? Bzw. ist das überhaupt relevant?
- p_M oder p_K im ILP
- wie kann p_K modeliert werden?

Besprechung am 06.05.

Nächste Schritte

- gleichungen linearisieren, wenn möglich
- p_M^2 und p_M^3 modellieren
- space consumption modellieren

Fragen

- Ist der Zugriff auf R in den Unterschiedlichen Leveln relevant für die Laufzeit? - Denke schon, da dadurch höhere Level "bestraft" werden
- Ist die Zugriffszeit auf R bedeutend abhängig von m und t? - Wahrscheinlich nicht entscheidend aber könnte ebenfalls modelliert werden

Besprechung am 29.04.

Nächste Schritte

- ILP für Laufzeit und Größe überlegen
- tex studio installieren - done
- erstmal nur lookup betrachten + (streaming locate? testen obs funktioniert)
- nach dem timen nochmal über alle positionen iterieren, damit der compiler nichts wegoptimiert

- überprüfen, ob tatsächlicher locate/lookup linear abhängig von minimizer verteilung ist
- in build.cpp gucken für implementierung
- streaming query lookup aus main - queries durch textstellen ersetzten, an denen minimizer mit frequenz i vorkommen
- überlegung: tatsächlich muss ja zunächst im Bitvektor und dann im Text nachgeschaut werden - zeiten variieren je nach system und die parameter können systemabhängig optimiert werden

Fragen

- BA Template? - kommt
- minimizer stats output - geklärt
- Warum kann ich *.../rshash$ .build/source/rshash –help* nicht ausführen aber *.../rshash/build/source$ ./rshash –help* schon? - fehlender backslash im PFad
- Unterscheidet sich mini_stats auf dem dev branch von mini_stats auf dem main branch? - nein
- was wäre eine sinnvolle Projektstruktur, wenn ich selber Code schreiben will? - es müsste möglich sein, programme aus rshash von BA directory aufzurufen Wäre es okay, wenn ich eine Kopie des rshash repositories erstelle und diese in meinem git_lab hochlade? - ja aber nicht nötig
- Fragen zu code:
  - vieles passiert bereits in mini_stats - ist das effizient gelöst? - ja
  - falls ja, kann ich den Code nutzten und anpassen? - ja
  - wäre es sinnvoll die minimizer komprimiert zu speichern? z.B. nur zwei bit pro Base oder gleich als Zahl? - jain, wird durch seqan bib bereits effizient gelöst
  - muss ich für jedes m eine Datei mit den minimizern anlegen oder reicht der Arbeitsspeicher, um das ganze in einer Datenstruktur zu behalten? - Arbeitsspreicher sollte vorerst ausreichen. Für sehr große Genome bzw. Sammlungen von vielen großen Genomen wären dateien nötig. Eine Datei wäre vielleicht trotzdem sinnvoll, um später damit zu arbeiten.